Practical DevOps Training Guide

NextJS + MongoDB + Docker Compose + Jenkins CI/CD + Ubuntu Deployment

---

Objective

This practical training is designed to help you understand:

- Docker fundamentals
- Docker Compose architecture
- Containerized NextJS deployment
- MongoDB container deployment
- Reverse proxy with Nginx
- Jenkins CI/CD pipeline
- GitHub integration
- Automated deployment flow
- Infrastructure as Code mindset
- Server migration strategy
- Production deployment concepts

By the end of this training, you will be able to:

- Deploy a fullstack application using Docker Compose
- Setup CI/CD automation with Jenkins
- Automatically deploy code changes from GitHub
- Move the entire ecosystem to another server easily
- Understand how modern DevOps deployment works

Final Architecture

```
GitHub Repository
        ↓
Git Push
        ↓
Jenkins Pipeline
        ↓
Build Docker Images
        ↓
Deploy with Docker Compose
        ↓
Ubuntu Server
 ├── Nginx
```

```
├── NextJS App
├── MongoDB
└── Docker Network
```

Recommended Infrastructure

Server Requirement

Minimum:

Component	Requirement
CPU	2 Core
RAM	4GB
Storage	40GB SSD
OS	Ubuntu 22.04

Recommended:

Component	Requirement
CPU	4 Core
RAM	8GB
Storage	80GB SSD

Phase 1 — Prepare Ubuntu Server

SSH into your Ubuntu server.

Update System

```
sudo apt update && sudo apt upgrade -y
```

Install Required Packages

```
sudo apt install -y
  curl
```

```
  git
  unzip
  ca-certificates
  gnupg
  lsb-release
```

Install Docker

Add Docker GPG Key

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Add Docker Repository

```
echo
  "deb [arch=$(dpkg --print-architecture)
  signed-by=/etc/apt/keyrings/docker.gpg]
  https://download.docker.com/linux/ubuntu
  $(lsb_release -cs) stable" |
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Install Docker Engine

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

Enable Docker

```
sudo systemctl enable docker
sudo systemctl start docker
```

Add Current User to Docker Group

```
sudo usermod -aG docker $USER
```

Reconnect SSH after this.

Verify Docker

```
docker --version
docker compose version
```

Phase 2 — Project Structure

Prepare your project structure like this:

```
my-project/
├── app/
│   └── NextJS Source
├── nginx/
│   └── default.conf
├── docker-compose.yml
├── Dockerfile
├── .env
└── Jenkinsfile
```

Phase 3 — Create Dockerfile

At project root:

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

RUN npm run build
```

```
FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app .

EXPOSE 3000

CMD ["npm", "start"]
```

Understanding This Dockerfile

Multi-stage Build

This is important.

Stage 1:

```
builder
```

Used for:

- dependency install
- compiling
- building

Stage 2:

```
production runtime
```

Only contains:

- built application
- required runtime

Benefits:

- smaller image
- more secure
- faster deployment

Phase 4 — Setup Nginx Reverse Proxy

Create:

```
nginx/default.conf
```

Nginx Config

```
server {
    listen 80;

    location / {
        proxy_pass http://nextjs:3000;

        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Important Concept

Notice:

```
nextjs:3000
```

This is NOT localhost.

Docker Compose automatically creates internal DNS.

Container names become hostnames.

Phase 5 — Setup Docker Compose

Create:

docker-compose.yml

Docker Compose Configuration

```
services:
  nextjs:
    build:
      context: .
    container_name: nextjs-app
    restart: always
    env_file:
      - .env
    depends_on:
      - mongodb
    networks:
      - app-network

  mongodb:
    image: mongo:7
    container_name: mongodb
    restart: always
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: password123
    volumes:
      - mongodb_data:/data/db
    networks:
      - app-network

  nginx:
    image: nginx:latest
    container_name: nginx
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - nextjs
    networks:
      - app-network

volumes:
  mongodb_data:
```

```
networks:
  app-network:
```

Understanding Docker Compose

nextjs Service

Builds your application container.

---

mongodb Service

Runs MongoDB database.

---

nginx Service

Public-facing reverse proxy.

---

volumes

```
mongodb_data:
```

Very important.

Without this:

- restarting container may lose database data

---

networks

All containers communicate securely internally.

Phase 6 — Environment Variables

Create:

```
.env
```

Example:

```
MONGODB_URI=mongodb://admin:password123@mongodb:27017
NODE_ENV=production
PORT=3000
```

Phase 7 — First Deployment

Build Containers

```
docker compose build
```

Start Stack

```
docker compose up -d
```

Verify Running Containers

```
docker ps
```

Expected:

```
nextjs-app
mongodb
nginx
```

Test Deployment

Open browser:

```
http://YOUR_SERVER_IP
```

Your NextJS app should appear.

Useful Docker Commands

View Logs

```
docker compose logs -f
```

Restart Services

```
docker compose restart
```

Stop Stack

```
docker compose down
```

Rebuild Application

```
docker compose up -d --build
```

Phase 8 — Install Jenkins

Install Java

```
sudo apt install openjdk-17-jdk -y
```

Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo
tee
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
sudo apt install jenkins -y
```

Enable Jenkins

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

Access Jenkins

```
http://SERVER_IP:8080
```

Get Initial Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Install Suggested Plugins

Inside Jenkins:

- Install suggested plugins
- Create admin user

Phase 9 — GitHub Integration

Install Git Plugin

Inside Jenkins:

```
Manage Jenkins
→ Plugins
→ Git Plugin
```

Add GitHub Credentials

Use:

```
Manage Jenkins
→ Credentials
```

Add:

- GitHub Token
- SSH Key

Phase 10 — Create Jenkins Pipeline

Create:

```
Jenkinsfile
```

Jenkins Pipeline Example

```
pipeline {
    agent any

    stages {
        stage('Clone Repository') {
            steps {
                git branch: 'main', url: 'YOUR_GITHUB_REPO'
            }
        }

        stage('Build Docker Images') {
            steps {
                sh 'docker compose build'
            }
        }
```

```
        stage('Deploy Containers') {
            steps {
                sh 'docker compose up -d'
            }
        }

        stage('Cleanup') {
            steps {
                sh 'docker image prune -f'
            }
        }
    }
}
```

Important CI/CD Concepts

Jenkinsfile

This is:

```
Pipeline as Code
```

Your deployment process becomes reproducible.

Build Stage

Creates fresh Docker images.

Deploy Stage

Updates running containers.

Cleanup Stage

Removes unused Docker images.

Important because Docker storage grows fast.

Phase 11 — Automatic Deployment

Inside Jenkins:

```
New Item
→ Pipeline
```

Select:

```
Pipeline script from SCM
```

Use:

- GitHub repository
- main branch

GitHub Webhook

Inside GitHub Repository:

```
Settings
→ Webhooks
→ Add Webhook
```

Payload URL:

```
http://YOUR_JENKINS_SERVER/github-webhook/
```

Content Type:

```
application/json
```

Flow After Setup

Now deployment becomes:

```
Git Push
→ GitHub Webhook
```

```
→ Jenkins Trigger
→ Docker Build
→ Container Deployment
→ Live Update
```

This is modern CI/CD.

Phase 12 — Production Improvements

Enable HTTPS

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx -y
```

Generate SSL Certificate

```
sudo certbot --nginx
```

Add Firewall

```
sudo ufw allow 80
sudo ufw allow 443
sudo ufw allow 22
sudo ufw enable
```

Phase 13 — MongoDB Backup Strategy

This is VERY important.

Create Backup

```
docker exec mongodb mongodump
  --username admin
  --password password123
  --archive=/dump.gz
  --gzip
```

Copy Backup From Container

```
docker cp mongodb:/dump.gz ./dump.gz
```

Restore Backup

```
docker cp ./dump.gz mongodb:/dump.gz
```

```
docker exec mongodb mongorestore
  --username admin
  --password password123
  --archive=/dump.gz
  --gzip
```

Phase 14 — Full Ecosystem Migration

This phase is extremely important.

This is where Docker Compose becomes powerful.

Scenario

You want to move:

```
Ubuntu Server A
→ Ubuntu Server B
```

with:

- NextJS app
- MongoDB
- Nginx
- configs
- deployment system

Migration Checklist

Copy:

```
Project Source
Docker Compose File
Nginx Config
Environment Variables
MongoDB Backup
```

Step 1 — Setup New Ubuntu Server

Repeat:

- Docker installation
- Docker Compose installation

Step 2 — Clone Repository

```
git clone YOUR_REPOSITORY
```

Step 3 — Copy Environment File

```
scp .env user@NEW_SERVER:/project/
```

Step 4 — Restore MongoDB Backup

Transfer backup:

```
scp dump.gz user@NEW_SERVER:/project/
```

Step 5 — Start Stack

```
docker compose up -d
```

Step 6 — Restore Database

```
docker cp dump.gz mongodb:/dump.gz
```

```
docker exec mongodb mongorestore
  --username admin
  --password password123
  --archive=/dump.gz
  --gzip
```

Important Lesson

Notice what happened.

You did NOT:

- manually install NodeJS
- manually install MongoDB
- manually configure Nginx
- manually configure runtime dependencies

Everything came from:

```
Dockerfile
Docker Compose
Infrastructure as Code
```

This is the real power of containers.

Phase 15 — Recommended Improvements

After mastering the basics, learn:

Docker Registry

Push images to:

- Docker Hub
- AWS ECR
- GitHub Container Registry

Blue-Green Deployment

Zero downtime deployment strategy.

Monitoring Stack

Learn:

- Prometheus
- Grafana
- Loki
- OpenTelemetry

Multi-environment Deployment

Separate:

```
Development
Staging
Production
```

Secrets Management

Avoid plain text passwords.

Learn:

- Docker Secrets

- Vault
- AWS Secrets Manager

Image Optimization

Learn:

- alpine images
- multistage builds
- layer caching

Important DevOps Mindset

Traditional deployment:

```
Server-centric
```

Modern deployment:

```
Container-centric
```

Advanced cloud-native deployment:

```
Cluster-centric
```

Final Learning Outcome

After completing this practical:

You will understand:

- Docker lifecycle
- Container networking
- Docker Compose orchestration
- CI/CD automation
- Jenkins pipeline
- GitHub integration
- Reverse proxy architecture
- Infrastructure as Code
- Migration strategy
- Modern deployment workflow

This foundation is enough to continue into:

- Kubernetes
- AI Ops
- Platform Engineering
- LLMOps
- Cloud-native systems
- Distributed architecture

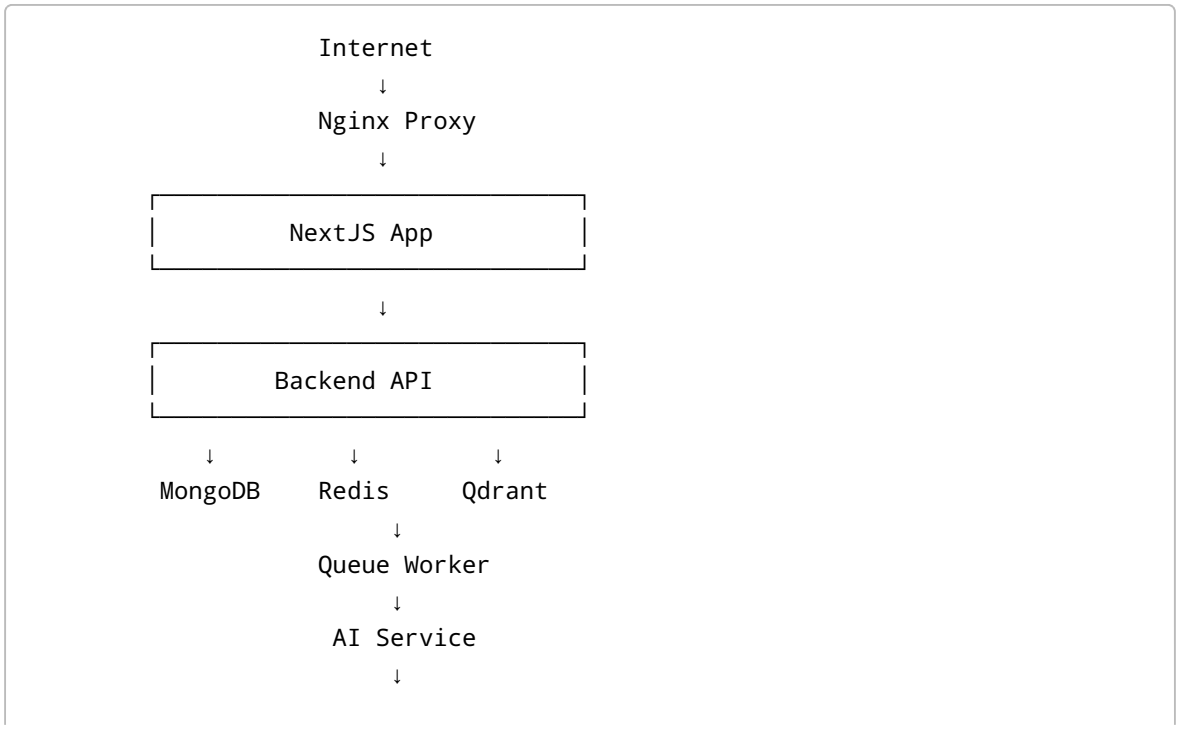
Phase 16 — Advanced Production Architecture

Now we upgrade the training into a more realistic AI-ready infrastructure.

You will learn:

- Fullstack microservice architecture
- AI service deployment
- Redis queue architecture
- Vector database deployment
- Monitoring stack
- Docker secret management
- Zero downtime deployment
- Production observability
- AI infrastructure concepts

Final Advanced Architecture



```
                    Ollama / LLM

_____________________________________

Monitoring Layer

Prometheus
Grafana
Loki
Node Exporter
cAdvisor

_____________________________________

CI/CD Layer

GitHub
↓
Jenkins
↓
Docker Compose Deployment
```

Advanced Project Structure

```
ai-platform/
├── frontend/
│   └── NextJS
├── backend/
│   └── Express/FastAPI/NestJS
├── worker/
│   └── Queue Worker
├── ai-service/
│   └── AI Processing Service
├── nginx/
├── monitoring/
│   ├── prometheus/
│   ├── grafana/
│   └── loki/
├── docker-compose.yml
├── docker-compose.prod.yml
├── .env
├── .env.production
├── secrets/
└── Jenkinsfile
```

Understanding The Architecture

Frontend

NextJS frontend.

Responsibilities:

- UI
- Authentication
- Dashboard
- Real-time updates
- File upload

Backend API

Handles:

- business logic
- JWT auth
- database access
- API gateway
- queue creation

Recommended:

- NestJS
- FastAPI
- Express

Redis Queue

Critical for AI systems.

Used for:

- async jobs
- AI processing
- email jobs
- notifications
- image processing
- long-running tasks

Without queue:

```
User waits for AI response
```

With queue:

```
User submits job
→ background worker processes
→ frontend polls/subscribes
```

Queue Flow

```
Frontend
   ↓
Backend API
   ↓
Redis Queue
   ↓
Worker Service
   ↓
AI Service
   ↓
MongoDB
```

Qdrant Vector Database

Very important for AI systems.

Used for:

- embeddings
- semantic search
- RAG systems
- AI memory
- document retrieval

AI Service

Separate AI processing service.

Responsibilities:

- embeddings

- AI inference
- document parsing
- OCR
- prompt orchestration
- LLM integration

Recommended:

- Python FastAPI
- LangChain
- LlamaIndex

Advanced Docker Compose Example

```
services:
  frontend:
    build: ./frontend
    restart: always
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build: ./backend
    restart: always
    env_file:
      - .env
    depends_on:
      - mongodb
      - redis
      - qdrant
    networks:
      - app-network

  worker:
    build: ./worker
    restart: always
    depends_on:
      - redis
      - ai-service
    networks:
      - app-network

  ai-service:
    build: ./ai-service
    restart: always
    networks:
      - app-network
```

```
  mongodb:
    image: mongo:7
    restart: always
    volumes:
      - mongodb_data:/data/db
    networks:
      - app-network

  redis:
    image: redis:7
    restart: always
    networks:
      - app-network

  qdrant:
    image: qdrant/qdrant
    restart: always
    volumes:
      - qdrant_data:/qdrant/storage
    networks:
      - app-network

  nginx:
    image: nginx:latest
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - frontend
    networks:
      - app-network

volumes:
  mongodb_data:
  qdrant_data:

networks:
  app-network:
```

Phase 17 — Monitoring Stack

Monitoring is CRITICAL.

Production systems without monitoring are blind.

Monitoring Architecture

```
Containers
   ↓
Metrics Collection
   ↓
Prometheus
   ↓
Grafana Dashboard
```

Monitoring Components

Component	Purpose
Prometheus	Metrics collection
Grafana	Dashboard visualization
Loki	Log aggregation
cAdvisor	Container metrics
Node Exporter	Server metrics

Add Monitoring Services

```yaml
  prometheus:
    image: prom/prometheus
    volumes:
      - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    networks:
      - app-network

  grafana:
    image: grafana/grafana
    ports:
      - "3001:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - app-network
```

```
  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    ports:
      - "8081:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    networks:
      - app-network
```

What You Should Monitor

Infrastructure Metrics

- CPU usage
- RAM usage
- Disk usage
- Network traffic

Container Metrics

- container restart count
- memory leak
- container health
- container CPU usage

Application Metrics

- API latency
- request count
- error rate
- websocket connections

AI Metrics

- token usage
- inference latency
- embedding latency
- queue size
- AI request failure rate

Grafana Dashboards

Create dashboards for:

- Server Overview
- Docker Containers
- Backend API Metrics
- Redis Queue Metrics
- AI Inference Metrics

Phase 18 — Docker Secrets Management

Hardcoding secrets is dangerous.

BAD:

```
MONGODB_PASSWORD=password123
```

Docker Secrets Concept

Instead of environment variables:

```
Secret stored securely
→ mounted into container
```

Create Docker Secret Files

```
mkdir secrets
```

Example Secret Files

```
secrets/
mongodb_password.txt
jwt_secret.txt
openai_api_key.txt
```

Example Secret Usage

```
services:
  backend:
    secrets:
      - mongodb_password
      - jwt_secret

secrets:
  mongodb_password:
    file: ./secrets/mongodb_password.txt

  jwt_secret:
    file: ./secrets/jwt_secret.txt
```

Access Secrets Inside Container

Secrets mounted at:

```
/run/secrets/
```

Example:

```
const fs = require('fs')

const password = fs.readFileSync('/run/secrets/mongodb_password', 'utf8')
```

Important Security Concepts

Never:

- commit secrets into Git
- expose .env publicly
- hardcode API keys

Recommended Production Secret Management

After Docker secrets, learn:

- Vault
- AWS Secrets Manager
- Doppler
- Kubernetes Secrets

Phase 19 — Zero Downtime Deployment

This is one of the most important production concepts.

The Problem

Normal deployment:

```
Stop old container
→ Start new container
```

Downtime occurs.

Better Strategy

```
Start new container
→ Health check passes
→ Switch traffic
→ Remove old container
```

Blue-Green Deployment Concept

```
Current Environment = Blue
New Deployment = Green
```

Flow:

```
Deploy Green
→ Validate
→ Switch Nginx traffic
→ Remove Blue
```

Zero Downtime Nginx Strategy

Example

```
nextjs-blue
nextjs-green
```

Nginx routes traffic to active deployment.

Health Check Example

Docker Compose:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3000"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Jenkins Zero Downtime Flow

```
Build New Image
→ Start Green Deployment
→ Run Health Check
→ Switch Nginx Upstream
→ Remove Blue Deployment
```

Phase 20 — Advanced Jenkins Pipeline

Upgrade Jenkins pipeline.

Advanced Jenkinsfile

```
pipeline {
    agent any

    environment {
        COMPOSE_PROJECT_NAME = "ai-platform"
    }

    stages {

        stage('Clone Repository') {
            steps {
                git branch: 'main', url: 'YOUR_GITHUB_REPO'
            }
        }

        stage('Build Images') {
            steps {
                sh 'docker compose build'
            }
        }

        stage('Run Tests') {
            steps {
                sh 'docker compose run backend npm test'
            }
        }

        stage('Deploy') {
            steps {
                sh 'docker compose up -d'
            }
        }

        stage('Health Check') {
            steps {
                sh 'curl -f http://localhost/health || exit 1'
            }
        }

        stage('Cleanup') {
            steps {
                sh 'docker image prune -f'
            }
        }
    }
}
```

Phase 21 — AI Developer Mindset

Modern AI developers are NOT just:

```
Prompt Engineers
```

Real AI developers understand:

- infrastructure
- deployment
- observability
- GPU resources
- vector databases
- queues
- async systems
- scaling
- inference architecture
- distributed systems

Real AI Production Flow

```
User Uploads File
        ↓
Frontend
        ↓
Backend API
        ↓
Redis Queue
        ↓
Worker
        ↓
AI Service
        ↓
Embedding Generation
        ↓
Qdrant Storage
        ↓
RAG Search
        ↓
LLM Response
        ↓
Frontend Display
```

Important Future Learning Path

After mastering this:

Infrastructure

Learn:

- Kubernetes
- Helm
- Terraform
- Ansible

AI Infrastructure

Learn:

- Ollama
- vLLM
- CUDA
- GPU scheduling
- distributed inference

AI Stack

Learn:

- LangChain
- LlamaIndex
- DSPy
- Haystack

Observability

Learn:

- OpenTelemetry
- distributed tracing
- AI observability
- prompt evaluation

Cloud Infrastructure

Learn:

- AWS ECS
- EKS
- GCP GKE
- Azure AKS

Recommended Final Practice

Build this complete system:

```
AI Knowledge Base Platform

Features:
- Authentication
- File Upload
- PDF Parsing
- OCR
- Embedding Generation
- Vector Search
- AI Chat
- Queue Processing
- Monitoring Dashboard
- CI/CD Automation
- Zero Downtime Deployment
- Backup & Recovery
- Multi-environment Deployment
```

This project will teach:

- Fullstack engineering
- AI engineering
- DevOps
- Platform engineering
- AI Ops
- LLMOps
- Cloud-native architecture

Final Mindset

You are no longer just learning:

```
How to deploy a website
```

You are learning:

```
How modern distributed AI systems are engineered and operated
```

That is the actual road toward becoming a real AI infrastructure developer.